

# Selección de variables con redes neuronales

Comparación de representaciones para regresión y clasificación

**Curso:** Inteligencia Artificial • **Semana y clase:** semana 6, clase 6

**Tema:** ingeniería y selección de características.

**Entorno de trabajo:** Jupyter Notebook con Python y `scikit-learn`.

**Modalidad:** comparación reproducible de redes neuronales con distintas selecciones de variables.

Las actividades se desarrollarán en notebooks de Jupyter con Python y `scikit-learn`. En ambos casos se deberá comparar una RNA entrenada con todas las variables contra la misma arquitectura entrenada con un subconjunto seleccionado. La selección de variables debe realizarse usando solamente datos de entrenamiento.

## Índice

|                                                                    |   |
|--------------------------------------------------------------------|---|
| 1. Consigna 1. Regresión temporal del consumo de electrodomésticos | 2 |
| 2. Consigna 2. Clasificación no temporal de diagnósticos           | 4 |
| 3. Criterio común de comparación                                   | 5 |

# 1. Consigna 1. Regresión temporal del consumo de electrodomésticos

## Dataset

**Appliances Energy Prediction**, UCI Machine Learning Repository. Contiene 19.735 registros tomados cada 10 minutos, 28 variables predictoras y el consumo **Appliances** como objetivo en Wh.

- **Fuente:** <https://archive.ics.uci.edu/dataset/374/appliances-energy-prediction>
- **Licencia:** CC BY 4.0.

## Objetivo

Explorar las variables ambientales y temporales, seleccionar un subconjunto informativo y ajustar una red neuronal artificial para predecir el consumo de electrodomésticos.

## Actividades

1. Descargar los datos, ordenar los registros por **date** y describir las variables.
2. Transformar **date** en atributos utilizables, como hora y día de la semana. No ingresar la fecha como texto en la red.
3. Explorar asociaciones con **Appliances**, redundancias entre sensores y el comportamiento de **rv1** y **rv2**.
4. Proponer al menos dos subconjuntos de variables mediante técnicas como correlación, información mutua, importancia por permutación o regularización.
5. Comparar, con la misma arquitectura, una RNA con todas las variables y otra con el subconjunto elegido.
6. Separar entrenamiento, validación y prueba respetando el orden temporal. No utilizar la prueba para seleccionar variables.
7. Evaluar con MAE, RMSE y  $R^2$ . Justificar la selección final y discutir estabilidad, redundancia y posibles fugas.

## Código inicial: descarga y preparación

```
%pip install ucimlrepo

import numpy as np
import pandas as pd
from ucimlrepo import fetch_ucirepo

energy = fetch_ucirepo(id=374)

X_raw = energy.data.features.copy()
y_raw = energy.data.targets.squeeze().astype(float)

df = X_raw.copy()
df["Appliances"] = y_raw.to_numpy()
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values("date").reset_index(drop=True)

# Atributos temporales básicos
df["hour_sin"] = np.sin(2 * np.pi * df["date"].dt.hour / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["date"].dt.hour / 24)
df["day_of_week"] = df["date"].dt.dayofweek
```

```

y = df.pop("Appliances")
X = df.drop(columns="date")

print(X.shape)
display(X.head())

```

## Código inicial: RNA de regresión

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Partición cronológica 70 % / 15 % / 15 %
n = len(X)
fin_train = int(0.70 * n)
fin_val = int(0.85 * n)

X_train, y_train = X.iloc[:fin_train], y.iloc[:fin_train]
X_val, y_val = X.iloc[fin_train:fin_val], y.iloc[fin_train:fin_val]
X_test, y_test = X.iloc[fin_val:], y.iloc[fin_val:]

# Comenzar con todas y reemplazar después por el subconjunto seleccionado
selected_features = list(X.columns)

rna_reg = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPRegressor(
        hidden_layer_sizes=(64, 32),
        activation="relu",
        max_iter=800,
        shuffle=False,
        random_state=42
    ))
])

rna_reg.fit(X_train[selected_features], y_train)
pred_val = rna_reg.predict(X_val[selected_features])

mae = mean_absolute_error(y_val, pred_val)
rmse = np.sqrt(mean_squared_error(y_val, pred_val))
r2 = r2_score(y_val, pred_val)

print({"MAE": mae, "RMSE": rmse, "R2": r2})

```

El conjunto de prueba deberá evaluarse una sola vez, después de seleccionar variables y congelar la configuración.

## Entrega

- Notebook ejecutable.
- Ranking o análisis de relevancia y redundancia.
- Comparación entre todas las variables y los subconjuntos candidatos.
- Lista final de variables con justificación.
- Métricas de validación y prueba, más una conclusión breve.

## 2. Consigna 2. Clasificación no temporal de diagnósticos

### Dataset

**Breast Cancer Wisconsin (Diagnostic)**, UCI Machine Learning Repository. Contiene 569 observaciones, 30 características numéricas y la etiqueta **Diagnosis**: maligno (M) o benigno (B).

- **Fuente:** <https://archive.ics.uci.edu/dataset/17/breast-cancer-wisconsin-diagnostic>
- **Licencia:** CC BY 4.0.
- Uso exclusivamente educativo; los resultados no constituyen una herramienta clínica.

### Objetivo

Explorar las 30 características, identificar variables relevantes o redundantes y ajustar una red neuronal artificial para clasificar el diagnóstico.

### Actividades

1. Descargar los datos, verificar clases, escalas y correlaciones. Excluir cualquier identificador.
2. Analizar la redundancia entre medidas de radio, perímetro, área, textura y sus diferentes resúmenes.
3. Proponer al menos dos subconjuntos mediante técnicas como ANOVA, información mutua, RFE, regularización L1 o importancia por permutación.
4. Comparar, con la misma arquitectura y particiones, una RNA con las 30 variables y otra con el subconjunto elegido.
5. Realizar una partición estratificada. Ajustar escalado y selección usando solamente entrenamiento.
6. Evaluar con accuracy, precisión, recall, F1 y matriz de confusión. Prestar especial atención a los casos malignos no detectados.
7. Justificar la selección final considerando desempeño, estabilidad y cantidad de variables.

### Código inicial: descarga y preparación

```
%pip install ucimlrepo

import pandas as pd
from ucimlrepo import fetch_ucirepo

cancer = fetch_ucirepo(id=17)

X = cancer.data.features.copy()
X = X.drop(columns=["ID"], errors="ignore")
y = cancer.data.targets.squeeze().map({"B": 0, "M": 1})

print(X.shape)
print(y.value_counts())
display(X.head())
```

### Código inicial: RNA de clasificación

```
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score
)
```

```

from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Prueba final estratificada
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

# Validación estratificada dentro del conjunto de desarrollo
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev,
    test_size=0.25,
    stratify=y_dev,
    random_state=42
)

# Comenzar con todas y reemplazar después por el subconjunto seleccionado
selected_features = list(X.columns)

rna_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        hidden_layer_sizes=(32, 16),
        activation="relu",
        max_iter=1000,
        random_state=42
    ))
])

rna_clf.fit(X_train[selected_features], y_train)
pred_val = rna_clf.predict(X_val[selected_features])

print("Accuracy:", accuracy_score(y_val, pred_val))
print("F1:", f1_score(y_val, pred_val))
print("Matriz de confusión:\n", confusion_matrix(y_val, pred_val))
print(classification_report(y_val, pred_val, target_names=["Benigno", "Maligno"]))

```

El conjunto de prueba deberá permanecer reservado hasta finalizar la selección de variables y la configuración de la RNA.

## Entrega

- Notebook ejecutable.
- Análisis de correlación, relevancia y redundancia.
- Comparación entre las 30 variables y los subconjuntos candidatos.
- Lista final de variables con justificación.
- Métricas, matriz de confusión y conclusión breve.

## 3. Criterio común de comparación

En cada práctica, cambiar únicamente el conjunto de variables. La arquitectura, particiones, semilla, preprocesamiento y métricas deben permanecer constantes. De este modo, la diferencia observada puede atribuirse razonablemente a la selección de features y no a un cambio simultáneo del experimento.